Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 30/07/2026**

COURSE NAME: Computer Networks **CLASS: T. Y. B.Tech**

Experiment No. 4

CO/LO: Analyse various concerns at different layers in reference models using suitable techniques.

Aim: Write a program to implement Hamming code for error detection and correction

Theory:

Hamming code is an error-correcting code used to ensure data accuracy during transmission or storage. Hamming code detects and corrects the errors that can occur when the data is moved or stored from the sender to the receiver. This simple and effective method helps improve the reliability of communication systems and digital storage. It adds extra bits to the original data, allowing the system to detect and correct single-bit errors. It is a technique developed by Richard Hamming in the 1950s.

What is Redundant Bits?

Redundant bits are extra binary bits that are generated and added to the information-carrying bits of data transfer to ensure that no bits were lost during the data transfer. The number of redundant bits can be calculated using the following formula:

$2^r \geq m + r + 1$
where,

- m is the number of bits in input data
- r is the number of redundant bits.

Suppose the number of data bits is 7, then the number of redundant bits can be calculated as

$= 2^4 \geq 7 + 4 + 1$
Thus, the number of redundant bits is 4.

Types of Parity Bits

A parity bit is a bit appended to a data of binary bits to ensure that the total number of 1's in the data is even or odd. Parity bits are used for error detection. There are two types of parity bits:

- **Even Parity Bit:** In the case of even parity, for a given set of bits, the number of 1's are counted. If that count is odd, the parity bit value is set to 1, making the total count of occurrences of 1's an even number. If the total number of 1's in a given set of bits is already even, the parity bit's value is 0.
- **Odd Parity Bit:** In the case of odd parity, for a given set of bits, the number of 1's are counted. If that count is even, the parity bit value is set to 1, making the total count of

occurrences of 1's an odd number. If the total number of 1's in a given set of bits is already odd, the parity bit's value is 0.

Algorithm of Hamming Code

Hamming Code is simply the use of extra parity bits to allow the identification of an error.

Step 1: Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).

Step 2: All the bit positions that are a power of 2 are marked as parity bits (1, 2, 4, 8, etc).

Step 3: All the other bit positions are marked as data bits.

Step 4: Each data bit is included in a unique set of parity bits, as determined its bit position in binary form:

- Parity bit 1 covers all the bits positions whose binary representation includes a 1 in the least significant position (1, 3, 5, 7, 9, 11, etc).
- Parity bit 2 covers all the bits positions whose binary representation includes a 1 in the second position from the least significant bit (2, 3, 6, 7, 10, 11, etc).
- Parity bit 4 covers all the bits positions whose binary representation includes a 1 in the third position from the least significant bit (4–7, 12–15, 20–23, etc).
- Parity bit 8 covers all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit bits (8–15, 24–31, 40–47, etc).
- In general, each parity bit covers all bits where the bitwise AND of the parity position and the bit position is non-zero.

Step 5: Since we check for even parity set a parity bit to 1 if the total number of ones in the positions it checks is odd. Set a parity bit to 0 if the total number of ones in the positions it checks is even.

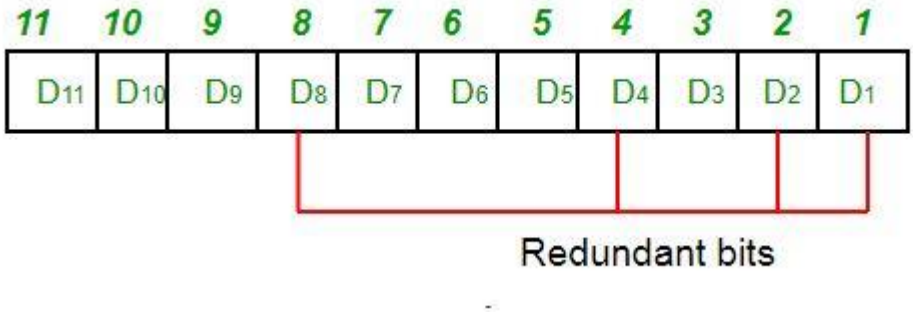
Position	R8	R4	R2	R1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1

R1 -> 1,3,5,7,9,11
R2 -> 2,3,6,7,10,11
R3 -> 4,5,6,7
R4 -> 8,9,10,11

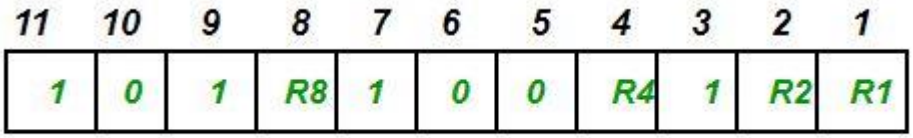
Determining The Position of Redundant Bits

A redundancy bits are placed at positions that correspond to the power of 2. As in the above example:

- The number of data bits = 7
- The number of redundant bits = 4
- The total number of bits = 7+4=11
- The redundant bits are placed at positions corresponding to power of 2 that is 1, 2, 4, and 8

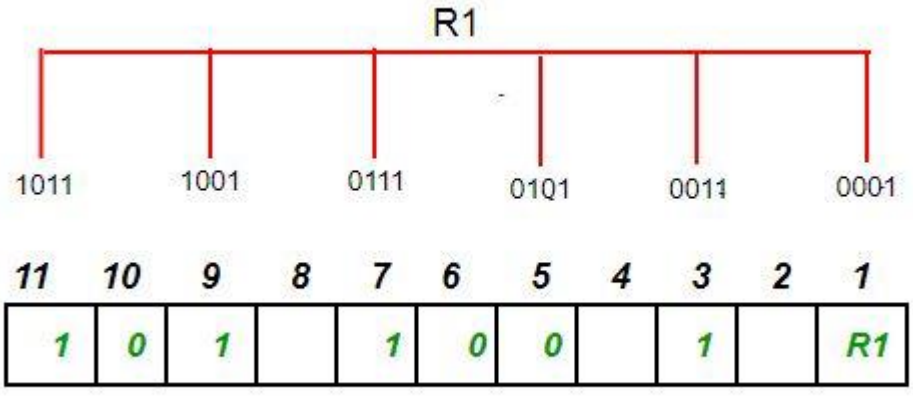


- Suppose the data to be transmitted is 1011001 from sender to receiver, the bits will be placed as follows:



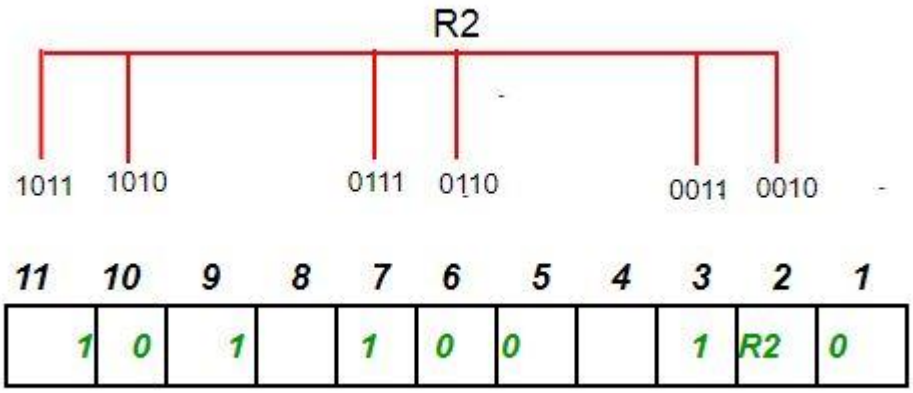
Determining The Parity Bits According to Even Parity

- R1 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the least significant position. R1: bits 1, 3, 5, 7, 9, 11

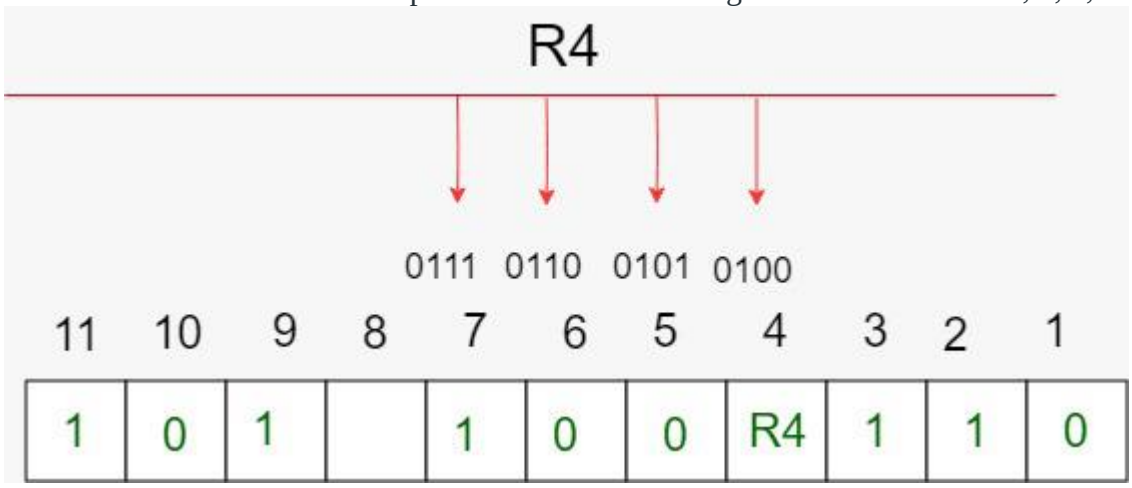


- To find the redundant bit R1, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R1 is an even number. So, the value of R1 (parity bit's value) = 0.
- R2 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the second position from the least significant bit. R2: bits 2,3,6,7,10,11

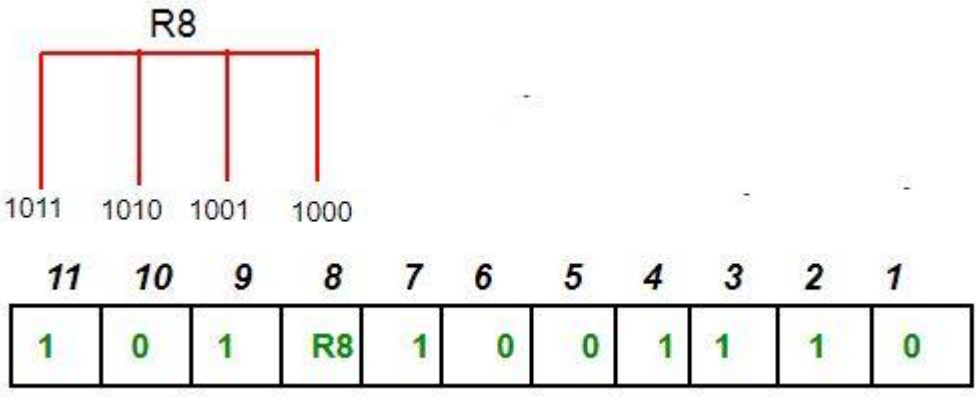
Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai



- To find the redundant bit R2, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R2 is odd the value of R2(parity bit's value)=1
- R4 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the third position from the least significant bit. R4: bits 4, 5, 6, 7

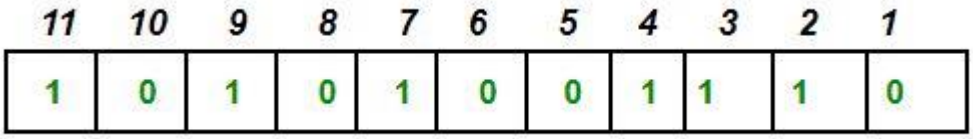


- To find the redundant bit R4, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R4 is odd so the value of R4(parity bit's value) = 1
- R8 bit is calculated using parity check at all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit. R8: bit 8,9,10,11



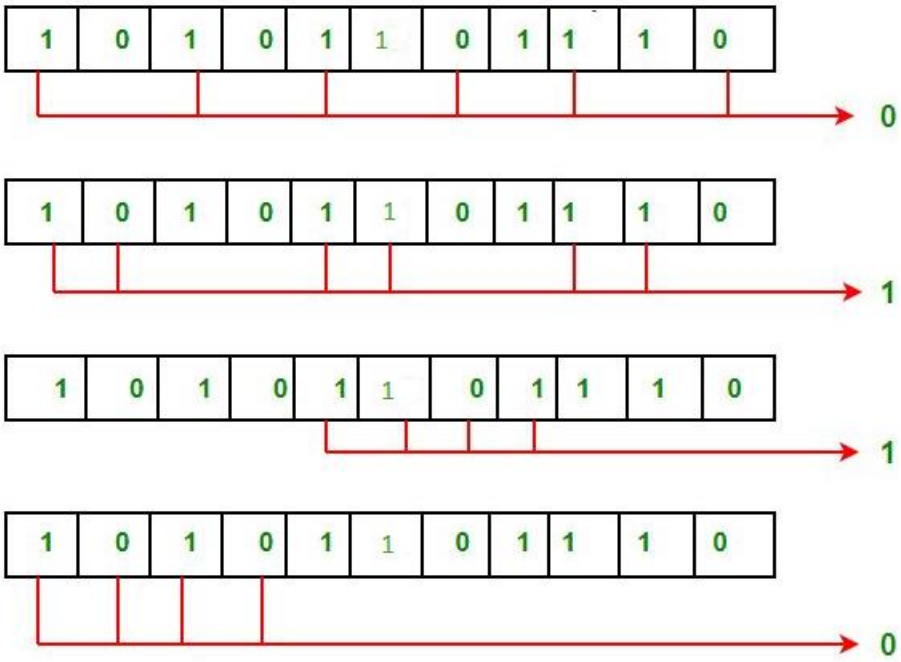
- To find the redundant bit R8, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R8 is an even number the value of R8(parity bit's

value)=0. Thus, the data transferred is:



Error Detection and Correction

Suppose in the above example the 6th bit is changed from 0 to 1 during data transmission, then it gives new parity values in the binary number:



For all the parity bits we will check the number of 1's in their respective bit positions.

- For R1: bits 1, 3, 5, 7, 9, 11. We can see that the number of 1's in these bit positions are 4 and that's even so we get a 0 for this.
- For R2: bits 2,3,6,7,10,11 . We can see that the number of 1's in these bit positions are 5 and that's odd so we get a 1 for this.
- For R4: bits 4, 5, 6, 7 . We can see that the number of 1's in these bit positions are 3 and that's odd so we get a 1 for this.
- For R8: bit 8,9,10,11 . We can see that the number of 1's in these bit positions are 2 and that's even so we get a 0 for this.
- The bits give the binary number 0110 whose decimal representation is 6. Thus, bit 6 contains an error. To correct the error the 6th bit is changed from 1 to 0.

EXERCISE

Your program should do the following :

1. Read the PDF File as binary data.
2. Split the Data into 16/32-bit Blocks.
3. Encode Each Block with Hamming Code:
4. Add parity bits
5. Simulate Transmission:
    a. Optionally introduce random single-bit errors.
6. Decode and Check Each Block:
    a. Detect and correct single-bit errors.
    b. Track number of errors and corrections.

Generate a Summary Table.

Code:

```
import random


def required_parity_bits(m):
    r = 0
    while 2 ** r < m + r + 1:
        r += 1
    return r


def hamming_encode(data):
    m = len(data)
    r = required_parity_bits(m)
    n = m + r
    code = [0] * (n + 1)

    j = 0

    for i in range(1, n + 1):
        if i & (i - 1):
            code[i] = data[j]
            j += 1

    for p in range(r):
        pos = 2 ** p
        parity = 0

        for i in range(1, n + 1):
            if i & pos:
                parity ^= code[i]

        code[pos] = parity

    return code[1:]
```

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

```
def hamming_decode(received):
    n = len(received)
    code = [0] + received
    syndrome = 0
    pos = 1

    while pos <= n:
        parity = 0

        for i in range(1, n + 1):
            if i & pos:
                parity ^= code[i]

        if parity:
            syndrome += pos

        pos *= 2

    corrected = False

    if syndrome != 0 and syndrome <= n:
        code[syndrome] ^= 1
        corrected = True

    data = []

    for i in range(1, n + 1):
        if i & (i - 1):
            data.append(code[i])

    return data, syndrome, corrected


def bytes_to_bits(data):
    bits = []

    for byte in data:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)

    return bits


def bits_to_bytes(bits):
    result = bytearray()

    for i in range(0, len(bits), 8):
```

```
        value = 0

        for bit in bits[i:i + 8]:
            value = (value << 1) | bit

        result.append(value)

    return bytes(result)


def split_blocks(bits, size):
    blocks = []

    for i in range(0, len(bits), size):
        block = bits[i:i + size]

        if len(block) < size:
            block += [0] * (size - len(block))

        blocks.append(block)

    return blocks


def transmit(bits, probability):
    bits = bits[:]

    if random.random() < probability:
        position = random.randrange(len(bits))
        bits[position] ^= 1
        return bits, position

    return bits, -1


def process_pdf(filename, block_size, error_probability):
    with open(filename, "rb") as file:
        original = file.read()

    original_bits = bytes_to_bits(original)
    blocks = split_blocks(original_bits, block_size)

    encoded_blocks = [
        hamming_encode(block)
        for block in blocks
    ]
```

```
    transmitted_blocks = []
    error_blocks = []

    for block_number, block in enumerate(encoded_blocks, 1):
        transmitted, error_position = transmit(
            block,
            error_probability
        )

        transmitted_blocks.append(transmitted)

        if error_position != -1:
            error_blocks.append(
                (block_number, error_position + 1)
            )

    decoded_bits = []
    detected_errors = 0
    corrected_errors = 0

    for block in transmitted_blocks:
        data, syndrome, corrected = hamming_decode(block)

        if syndrome != 0:
            detected_errors += 1

        if corrected:
            corrected_errors += 1

        decoded_bits.extend(data)

    decoded_bits = decoded_bits[:len(original_bits)]

    recovered = bits_to_bytes(decoded_bits)
    recovered = recovered[:len(original)]

    output_file = "recovered_sample_" + str(block_size) + ".pdf"

    with open(output_file, "wb") as file:
        file.write(recovered)

    return (
        len(blocks),
        len(error_blocks),
        detected_errors,
        corrected_errors,
        error_blocks
```

```
    )


if __name__ == "__main__":

    input_file = "sample.pdf"
    error_probability = 0.1

    for serial_no, frame_size in enumerate([16, 32], 1):

        total_blocks, error_count, detected, corrected, error_blocks =
process_pdf(
            input_file,
            frame_size,
            error_probability
        )

        print()
        print("Frame Size:", frame_size)
        print("Total Blocks:", total_blocks)
        print("Blocks with Error:", error_count)
        print("Errors Detected:", detected)
        print("Errors Corrected:", corrected)

        print()
        print("First 20 Error Blocks:")

        for block, bit in error_blocks[:20]:
            print(
                "Block", block,
                "-> Bit", bit
            )

        print()
        print(
            serial_no,
            frame_size,
            total_blocks,
            error_count
        )
```

Output:

```
First 20 Error Blocks:
Block 2 -> Bit 15
Block 4 -> Bit 12
Block 15 -> Bit 2
Block 26 -> Bit 17
Block 30 -> Bit 2
Block 49 -> Bit 4
Block 50 -> Bit 1
Block 63 -> Bit 9
Block 79 -> Bit 18
Block 90 -> Bit 3
Block 95 -> Bit 18
Block 111 -> Bit 19
Block 119 -> Bit 3
Block 144 -> Bit 14
Block 163 -> Bit 21
Block 166 -> Bit 4
Block 172 -> Bit 2
Block 183 -> Bit 17
Block 187 -> Bit 12
Block 201 -> Bit 8

1 16 144909 14596

Frame Size: 32
Total Blocks: 72455
Blocks with Error: 7240
Errors Detected: 7240
Errors Corrected: 7240

First 20 Error Blocks:
Block 16 -> Bit 31
Block 19 -> Bit 18
Block 35 -> Bit 22
Block 59 -> Bit 26
Block 81 -> Bit 24
Block 88 -> Bit 32
Block 91 -> Bit 11
Block 95 -> Bit 33
Block 115 -> Bit 10
Block 116 -> Bit 2
Block 134 -> Bit 37
Block 139 -> Bit 28
Block 140 -> Bit 32
Block 151 -> Bit 27
Block 154 -> Bit 17
Block 158 -> Bit 31
Block 163 -> Bit 25
Block 164 -> Bit 9
Block 177 -> Bit 10
Block 180 -> Bit 15

2 32 72455 7240
```

Observation:

Sr. No	Frame Size	Total Blocks	Blocks with Error
1	16	144909	14596
2	32	72455	7240

Conclusion:
The experiment successfully demonstrated the use of Hamming Code for error detection and correction in PDF data transmission. The PDF data was divided into 16-bit and 32-bit blocks, parity bits were added, and single-bit transmission errors were introduced randomly. The Hamming decoder successfully detected and corrected the errors, and the recovered PDF matched the original data.

Observation Questions:

1. For a 7-bit Hamming code 1011010, determine if there is an error. If yes, find the error position and correct it.
2. You received the following 9-bit Hamming code: 101100111 Assuming even parity,
   a) Check whether the received code contains any error.
   b) If an error exists, find the bit position of the error and correct it.
   c) Extract the original 5-bit data from the corrected code.

Reference:

[1] www.geeksforgeeks.org